

Model Prediction

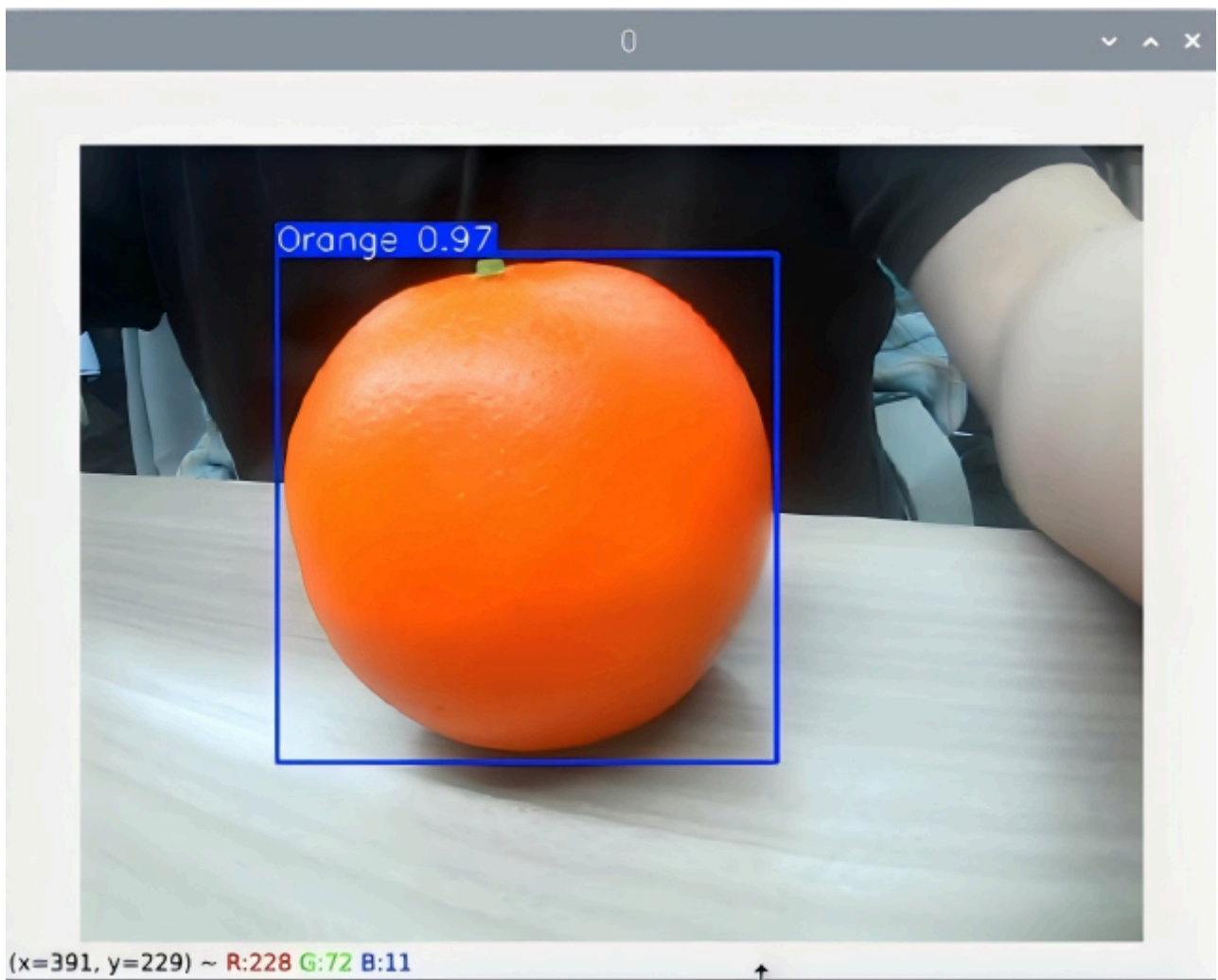
Model Prediction

1. CLI Usage
2. Python Usage
 - 2.1 USB Camera
 - [Result Preview](#)
 - 2.2 CSI Camera
 - [Result Preview](#)

1. CLI Usage

CLI currently only supports calling USB cameras. For CSI camera users, you can directly modify the previous Python code to call the ncnn model!

```
cd /home/sunrise/ultraalytics26/data/yahboom_data/orange_data/runs/detect/train/weights
yolo predict model='best_ncnn_model' source=0 save=False show # For multiple cameras,
change the number after source
```



2. Python Usage

Use Python to call USB camera and CSI camera to recognize oranges.

2.1 USB Camera

Use best_ncnn_model to predict camera feed:

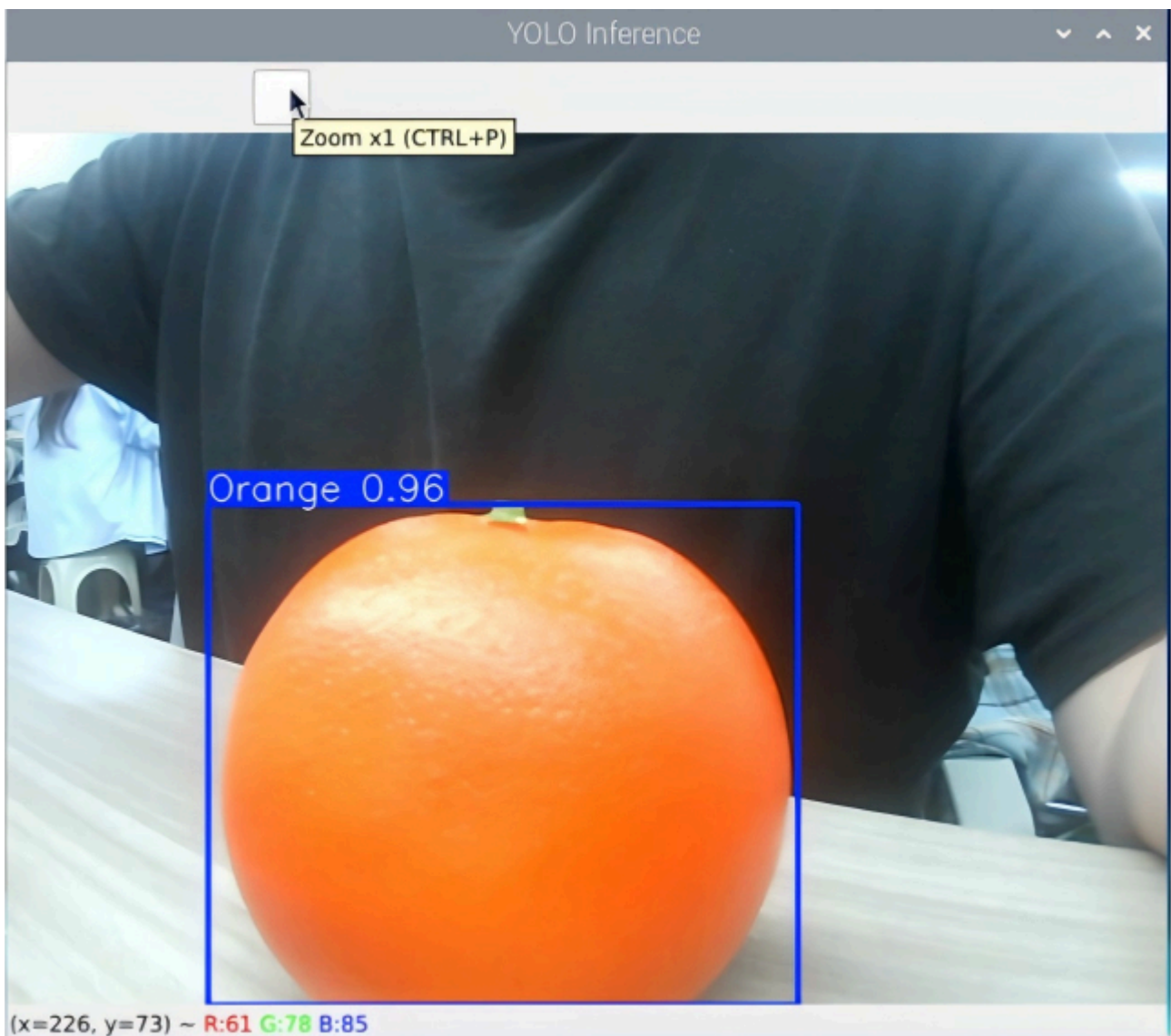
```
cd /home/sunrise/ultraalytics26/yahboom_demo
```

Run code: Click the preview screen, press q key to terminate the program!

```
python3 06.orange_camera_usb.py
```

Result Preview

yolo recognition output video location: /home/sunrise/ultraalytics26/output/



Example code:

```
import cv2
```

```

from ultralytics import YOLO

# Load the YOLO model
# model =
YOLO("/home/sunrise/ultralytics26/data/yahboom_data/orange_data/runs/detect/train/weights/best.pt")
model =
YOLO("/home/sunrise/ultralytics26/data/yahboom_data/orange_data/runs/detect/train/weights/best_ncnn_model")

# Open the camera
cap = cv2.VideoCapture(0)
cap.set(6, cv2.VideoWriter_fourcc('M', 'J', 'P', 'G'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/home/sunrise/ultralytics/ultralytics/output/06.orange_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v' depending on your platform
out = cv2.VideoWriter(output_path, fourcc, 30, (640, 480))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture and writer objects, and close the display window

```

```
cap.release()  
out.release()  
cv2.destroyAllWindows()
```

2.2 CSI Camera

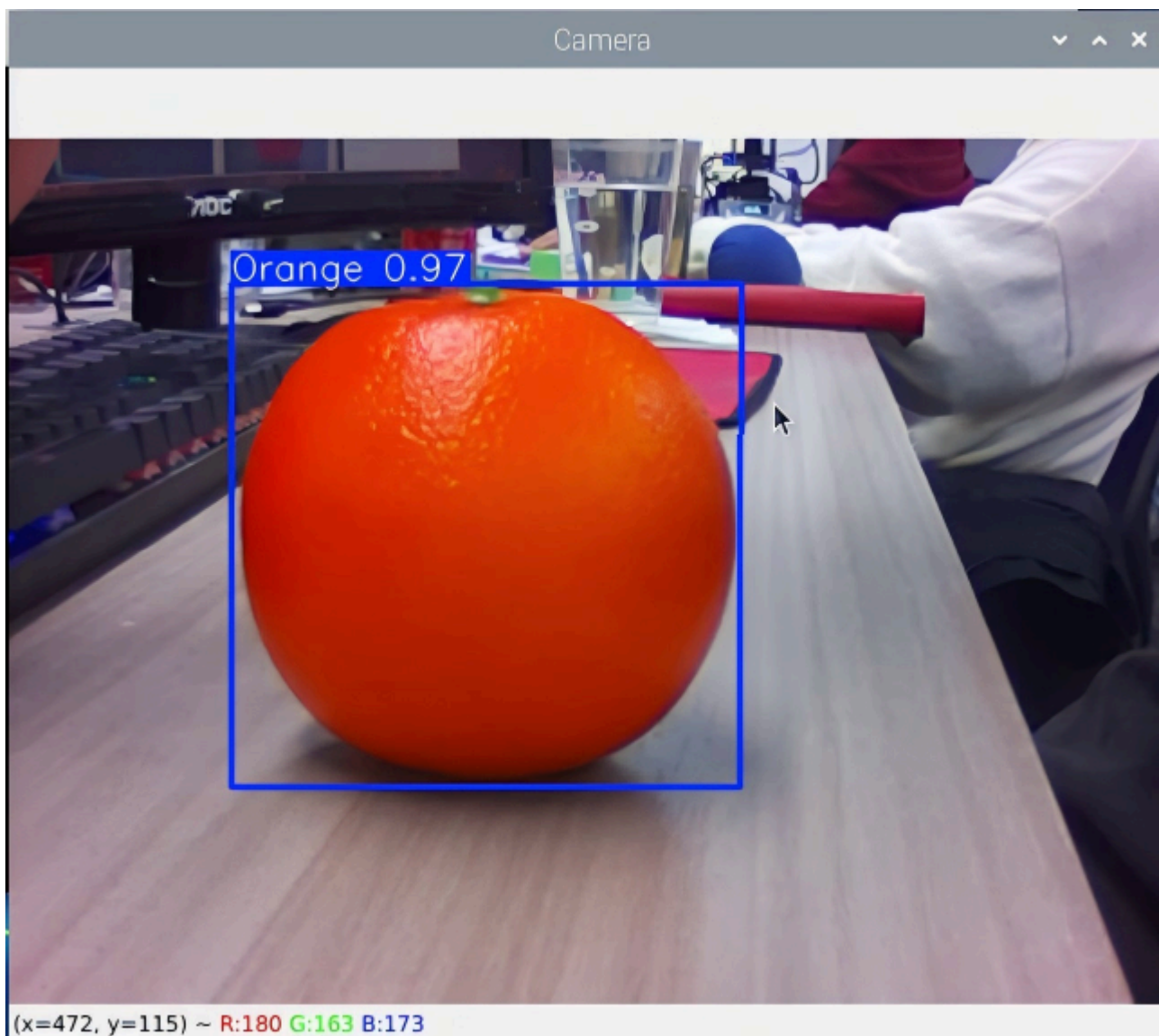
Use best_ncnn_model to predict camera feed:

```
cd /home/sunrise/ultraalytics26/yahboom_demo
```

Run code: Click the preview screen, press q key to terminate the program!

```
python3 06.orange_camera_csi.py
```

Result Preview



Example code:

```
import cv2
```

```

import sys
from hobot_vio import libsrcampy
import numpy as np

from ultralytics import YOLO

fps = 30
w = 1920
h = 1080

cam = libsrcampy.Camera()
ret = cam.open_cam(0, 0, fps, w, h)

# Load the YOLO model
# model =
YOLO("/home/sunrise/ultralytics26/data/yahboom_data/orange_data/runs/detect/train/weights/best.pt")
model =
YOLO("/home/sunrise/ultralytics26/data/yahboom_data/orange_data/runs/detect/train/weights/best_ncnn_model")

# Set up video output
output_path = "/home/sunrise/ultralytics26/output/06.orange_camera_csi.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out = cv2.VideoWriter(output_path, fourcc, 30, (640, 480))

# Loop through the video frames
while True:
    img_bytes = cam.get_img(1)
    yuv = np.frombuffer(img_bytes, dtype=np.uint8).reshape(-1, w)
    frame = cv2.cvtColor(yuv, cv2.COLOR_YUV2BGR_NV12)

    # Run YOLO11 inference on the frame
    results = model(frame)

    # Visualize the results on the frame
    annotated_frame = results[0].plot()

    # Display the resulting frame
    cv2.imshow("Camera", annotated_frame)

    # Break the loop if 'q' is pressed
    if cv2.waitKey(1) == ord("q"):
        break

# Release resources and close windows

out.release()
cv2.destroyAllWindows()

```

